本地代码安全审核技术调研与实施流程

本地化工具扫描、人工审核与整改核验

目录

1. 1. 调研目标
2. 2. 服务说明
3. 3. 整体流程
4. 4. 审核准备
5. 5. 从 Git 获取代码
6. 6. 全量审核
7. 7. 增量审核
8. 8. 代码具体怎么审
9. 9. 工具复核与人工确认
10. 10. 问题去重与分级
11. 11. 出具审核报告
12. 12. 整改与核验
13. 13. 双方职责
14. 14. 交付物
15. 15. 方法选择依据
16. 16. 常见问题说明

1. 调研目标

本次调研围绕以下问题明确实施方法：

- 代码如何从 Git 仓库拉取并固定审核版本。
- 全量审核和增量审核的区别。
- 代码具体怎么检查，人工审核体现在哪里。
- 问题如何确认、出具报告、整改和核验。
- 审核结论的适用范围和双方职责。

2. 服务说明

代码安全审核由审核人员在约定环境中，对客户指定版本的代码开展安全分析。

审核人员先了解项目架构、业务流程和权限模型，再通过规则检查发现线索，对高风险模块和疑似问题进行人工代码阅读、调用链分析和必要验证。工具复核可辅助筛除满足严格条件的高可信误报，人工结论始终优先；正式报告只保留需要跟踪处置的问题。

实施方案概述：

> 本项目提供人工主导的代码安全审核服务。审核人员从客户授权的 Git 仓库拉取指定版本代码，通过全量或增量检查、重点代码抽查和调用链分析确认安全问题，给出整改建议，并在修复后进行核验。

3. 整体流程



下载高清流程图 · 查看矢量版

流程分为三个阶段：

阶段	核心工作	阶段结果
审核准备	明确范围、取得 Git 权限、固定代码版本	审核范围和版本基线
代码审核	全量或增量检查、人工分析、问题确认	正式问题清单和审核报告
风险处置	研发修复、人工核验，或由客户审批接受风险	核验结论和处置记录

4. 审核准备

项目开始前，双方确认以下信息：

确认项	内容
代码范围	仓库、分支、模块、主要语言和框架
审核版本	当前审核分支或 Commit
审核方式	全量、增量或整改核验

确认项	内容
增量基线	基线分支或上次已审核 Commit
重点风险	权限、数据、文件、命令、密钥等
排除范围	第三方库、生成代码、测试代码等
交付要求	报告格式、交付时间、整改周期

客户宜提供系统架构、角色权限、关键接口和敏感数据说明。这些资料有助于审核人员判断越权、跨租户访问和业务流程绕过问题。

5. 从 Git 获取代码

5.1 客户需要提供

- Git 仓库地址。
- 只读访问权限或临时授权凭据。
- 需要审核的分支或 Commit。
- 增量审核使用的基线分支或 Commit。
- 子模块、Git LFS 等特殊代码获取说明。

5.2 审核人员执行

1. 在约定的审核环境中自行拉取代码。
2. 校验目标分支和 Commit 是否与客户要求一致。
3. 记录仓库、分支、完整 Commit、拉取时间和审核批次。
4. 以固定 Commit 开展审核，避免审核期间代码变化影响结论。
5. 后续增量审核前更新仓库，并重新记录当前版本和基线版本。

5.3 源码安全要求

- 代码只保存在双方约定的设备、目录或客户内网环境。
- Git 权限遵循最小授权原则，项目结束后回收临时凭据。
- 未经客户书面授权，不向第三方网站、SaaS 或工具服务提交源码。
- 本地复核可以查看必要的真实代码上下文；报告离开受控环境前，应按约定处理密码、令牌、私钥和其他敏感数据。
- 按约定期限归档或清理代码副本、临时文件和审核证据。

6. 全量审核

全量审核适用于首次审核、重大版本上线、系统重构或原基线不可信的场景。

执行方式：

1. 使用双方确认的规则集，检查纳入范围且受支持的文件。
2. 梳理接口入口、认证授权、数据流和敏感操作。
3. 人工审查高风险模块和关键业务流程。
4. 对疑似问题追踪调用链并确认风险条件。
5. 对问题去重、分级，形成正式清单。

全量审核不等于人工逐行阅读全部文件。报告应明确规则覆盖范围、人工抽查模块、排除项和未覆盖内容。

7. 增量审核

增量审核适用于日常迭代、功能分支合入、紧急修复和整改复查。

7.1 基线选择

场景	推荐基线
功能分支审核	功能分支与目标分支的共同祖先
日常版本迭代	上一次已经审核的 Commit
整改核验	上次审核报告对应的 Commit
紧急修复	修复前的生产版本或双方指定 Commit

分支名称可能变化，最终应记录实际比较的 Commit。

7.2 检查范围

- 新增、修改和删除的代码。
- 变更代码涉及的上下游调用。
- 受公共组件、权限方法或配置变化影响的代码。
- 新增依赖和依赖版本变化。
- 上次问题在当前版本中的状态。

增量审核不能只看差异行。例如公共权限方法发生变化时，还要检查所有受影响的接口。

8. 代码具体怎么审

8.1 审核顺序

1. 阅读项目目录、构建文件和主要配置。
2. 识别接口、任务、消息和文件处理入口。
3. 梳理认证、授权、租户隔离和敏感数据流。
4. 通过规则检查定位风险线索。
5. 人工阅读重点代码并追踪调用链。
6. 核查输入、权限、防护、可达性和影响。
7. 去除误报和重复线索，形成正式问题。

8.2 使用工具

本地审核当前使用以下开源工具，并由审核人员对结果进行复核：

工具	主要用途	当前使用范围
Semgrep	静态代码规则检查	使用社区通用规则、本地 OWASP/CWE 规则和项目自定义规则，定位注入、危险函数、鉴权缺陷、不安全配置等风险线索
Gitleaks	凭据泄露检查	检查源码和配置文件中的 API Key、Token、私钥及其他疑似硬编码凭据；当前针对审核快照目录，不代表覆盖完整 Git 提交历史
Trivy Secret	敏感信息补充检查	对文件系统快照进行密钥与敏感配置检查，与 Gitleaks 交叉补充；当前未将其表述为完整的软件成分分析能力

三个工具分别产生独立执行记录。工具不可用、超时或执行失败时，批次应标记为部分完成并保留日志，不得把未执行解释为未发现问题。规则版本、工具版本、执行参数和排除范围应随审核批次记录，以便复查。

8.3 人工重点检查

风险领域	重点内容
身份与权限	登录、令牌、接口权限、数据权限、租户隔离
输入与执行	参数校验、反序列化、表达式、命令和脚本
数据安全	SQL、敏感字段、日志、输出脱敏
文件安全	上传、下载、路径拼接、压缩解压

风险领域	重点内容
外部通信	URL、重定向、证书校验、内部服务访问
密码密钥	硬编码凭据、弱算法、密钥存储
依赖配置	高风险依赖、调试配置、不安全默认值

8.4 人工复核判断

每个疑似问题需要回答：

- 输入是否能够被外部人员或低权限用户控制?
- 代码路径是否真实可达，需要什么身份和权限?
- 是否已经存在有效的校验、编码、白名单或隔离措施?
- 问题可能影响哪些账户、数据、租户或系统?
- 多条线索是否属于同一根因?
- 建议的修复方式是否适合当前项目?

复核结果分为：

结论	处理方式
已确认问题	进入正式整改清单
待验证风险	单独列出，不计入已确认问题数量
误报或不适用	不进入正式整改清单
接受风险	保留问题，由客户审批并设定复审期限

9. 工具复核与人工确认

工具扫描完成后，先整理命中位置、必要源码上下文和复核建议，再由审核人员结合完整代码、调用链、权限与部署条件确认风险。

步骤	输出
工具扫描	风险线索、规则与代码位置
工具复核	初审、复审与综合建议，任务进度及失败原因
人工审核	确认风险、误报、接受风险或整改待验证，并记录人员与依据
整改核验	修复版本、核验结论和处置记录

台账保留全部候选问题。正式报告保留人工已确认的风险，或完整通过工具可信度门槛且非误报的结果；工具候选项仍标明实际人工审核状态。工具失败或证据不足时，应补充证据并重新复核。

启用外部复核工具时，必要真实源码证据会提交至配置接口，应符合项目约定。工具不会覆盖人工结论；报告生成不修改任何原始记录。

详细说明见工具复核与人工审核流程。

10. 问题去重与分级

同一根因、同一代码位置被多次识别时，只保留一个正式问题；同一根因影响多个位置时，在一个问题中列出全部影响位置。不同模块需要独立修改，或影响范围明显不同的，应分别记录。

风险等级由审核人员根据可利用性、所需权限、影响范围和数据敏感度综合判断：

等级	判断参考
严重	可能导致核心系统失陷、大范围敏感数据泄露或关键权限失控
高危	利用条件较现实，可能影响重要数据、账户或服务器
中危	需要特定条件，但仍存在明确安全影响
低危	影响有限、利用困难或属于安全加固项

11. 出具审核报告

正式报告建议包含：

- 审核范围、当前 Commit 和增量基线。
- 审核方法、规则范围和人工抽查范围。
- 风险概况和优先整改建议。
- 问题位置、必要的本地代码证据和人工分析。
- 问题成立条件、影响和整改建议。
- 排除项、未覆盖内容和审核限制。

整改报告列表与 HTML、Word、PDF 导出使用相同的筛选、去重和统计口径。报告保留人工确认的风险，或初审、复审均完整完成、各项可信度分别大于 65% 且非误报的工具结果。工具候选项仍标明实际人工审核状态；人工误报不进入报告。台账保留全部候选问题。报告生成是只读操作，不修改原始检查结果、工具记录或人工结论。

每项问题至少包含：

字段	内容
问题编号与名称	唯一编号和易理解的标题

字段	内容
风险等级	严重、高危、中危或低危
代码位置	仓库相对路径、方法、接口和行号
代码证据	必要的本地代码上下文；对外交付时按约定处理敏感值
人工分析	输入来源、调用链、防护和判断依据
成立条件与影响	所需权限、触发条件和影响范围
整改建议	适合当前项目的修改方向

12. 整改与核验

12.1 研发整改

研发人员按问题编号反馈：

- 修复方案和修改说明。
- 修复分支及完整 Commit。
- 影响范围和自测结果。
- 暂不修复时的原因、补偿措施和审批结论。

12.2 审核人员核验

审核人员重新拉取修复版本，并逐项检查：

1. 修复 Commit 是否包含对应修改。
2. 问题根因是否已经消除。
3. 是否存在旁路或其他相同调用。
4. 本次修复及其受影响调用是否引入明显的新风险。
5. 相关规则复查和必要验证是否符合预期。

规则不再提示不能单独作为核验通过的依据。

默认以代码和配置复查为主。需要编译、运行代码、执行验证用例或访问测试环境时，双方应提前确认授权范围、账号、数据和允许的操作。

技术核验结论包括：

结论	含义
通过	根因已消除，未发现明显绕过路径
未通过	原问题仍存在、修复不完整或可以绕过

结论	含义
待验证	需要环境、配置、数据或业务人员配合

如客户决定暂不整改，应由客户履行接受风险审批，并记录补偿措施、责任人和复审期限。审核方负责记录该处置决定，不代替客户作出风险接受决策。

13. 双方职责

客户	审核方
提供 Git 地址、只读权限和准确版本	按授权范围拉取并保护代码
提供架构、权限和业务说明	制定审核计划并执行检查
确认问题业务背景和整改计划	人工复核、去重、分级并给出建议
完成修复并提供修复 Commit	对修复版本开展核验
审批接受风险事项	记录证据、范围和核验结论

14. 交付物

交付物	内容	形式
代码安全审核报告	范围、方法、风险概况、问题和建议	PDF 或 Word
整改问题清单	问题、等级、责任人、状态和计划	表格
整改核验报告	修复版本、核验证据、结论和遗留项	PDF 或 Word

具体交付物、文件形式、交付时间、整改轮次和核验轮次，以合同或项目实施计划为准。

15. 方法选择依据

方法	选择原因
Git 拉取与固定 Commit	直接取得授权代码，明确审核版本，便于复查和比较
本地执行检查	在约定环境中处理源码与证据，便于控制访问范围
全量与增量结合	首次建立审核基线，后续重点分析代码变化及其影响
工具扫描与人工审核	工具定位线索并提供复核建议，人工结合业务、权限和调用链确认问题

方法	选择原因
整改后再次核验	检查修复是否消除根因，形成有证据的处置结论

16. 常见问题说明

Q1：这是人工审核还是自动扫描?

这是人工主导的代码安全审核。规则检查提供风险线索，审核人员负责业务理解、调用链分析、问题确认、风险分级和整改建议。

Q2：是否会人工逐行阅读所有代码?

不会做不符合实际的逐行承诺。规则检查覆盖双方确认的范围，人工审核重点覆盖高风险模块、关键业务流程和疑似问题。报告会列明实际人工抽查范围。

Q3：代码如何获取?

客户提供 Git 地址、只读权限和目标版本，审核人员在约定环境中自行拉取并记录完整 Commit。后续审核和核验也以明确 Commit 为准。

Q4：代码是否会上传到外部平台?

代码在约定环境中处理。未经客户书面授权，不向第三方网站、SaaS 或工具服务提交源码和审核证据。

Q5：增量审核为什么需要基线?

基线用于判断本次新增、持续存在和已经修复的问题。分支名称可能变化，因此需要记录实际参与比较的 Commit。

Q6：规则命中为什么没有写进报告?

规则可能因测试代码、不可达路径、固定常量或已有防护产生误报。人工确认的误报以及满足严格条件的工具高可信误报不会进入正式报告；冲突、证据不足或审核失败的结果仍需人工处理。

Q7：同一个问题在多个位置出现时如何计算?

同一根因通常作为一个问题，并列出全部影响位置；不同模块需要独立整改或影响不同的，则分别记录。

Q8：修复后规则不再提示，是否就算通过?

不一定。审核人员还需要确认根因已经消除，并检查关联调用和绕过路径。

Q9：暂时不能修复怎么办?

客户可以落实补偿措施并履行接受风险审批，同时明确责任人和复审期限。接受风险不等于问题已修复。

Q10：代码审核能否替代渗透测试?

不能完全替代。代码审核侧重实现逻辑和调用链，渗透测试侧重运行环境和真实攻击路径，两者关注点不同。